

Triggers, Partitioning & Full-Text Search

Module 7 • Automate change, scale history and search meaningfully.

MODULE

07

90 minutes

20 min Explain
15 min Demonstrate
40 min Hands-on
15 min Review

developer_lab continues

Module 7 adds behavior, scale and discovery

1

AUTOMATE

Use triggers for rules and audit records that must accompany every change.

2

ORGANIZE

Partition time-based history so maintenance and pruning stay manageable.

3

DISCOVER

Convert natural-language queries into fast, ranked catalogue results.

Each feature solves a different problem—choose it only when that problem exists.

By the end, learners can complete six tasks

- 01 Explain trigger timing and row scope
- 02 Build and test a stock audit trigger
- 03 Create range partitions with a default safety net
- 04 Prove partition pruning with EXPLAIN
- 05 Build a stored tsvector and GIN index
- 06 Search, rank and highlight product matches**

SUCCESS = AUDIT + PRUNING + RANKED SEARCH

A trigger runs because a table event occurred

EVENT

INSERT
UPDATE [OF columns]
DELETE
TRUNCATE

TIMING

BEFORE: validate or modify
AFTER: audit or notify
INSTEAD OF: redirect a view

SCOPE

FOR EACH ROW: once per row
FOR EACH STATEMENT: once per SQL statement

Trigger function and trigger are separate objects.

Choose trigger timing from the required effect

TIMING	MEANING	TYPICAL USE
BEFORE	Reject or adjust NEW before storage	Validation, normalization
AFTER	Use the final successful row values	Audit record, derived workflow
INSTEAD OF	Replace an operation on a view	Writable multi-table interface

AFTER triggers see the accepted row but cannot modify it.

OLD, NEW and TG_OP describe the firing event

```
IF TG_OP = 'UPDATE' THEN
  -- OLD is the previous row
  -- NEW is the proposed/final row
  IF OLD.stock_qty IS DISTINCT FROM NEW.stock_qty THEN
    ...
  END IF;
  RETURN NEW;
ELSIF TG_OP = 'DELETE' THEN
  RETURN OLD;
END IF;
```

ROW VALUES

INSERT has NEW. DELETE has OLD. UPDATE has both.

NULL-SAFE CHANGE

IS DISTINCT FROM detects changes even when one value is NULL.

A row-level trigger function must return the correct row value.

Create an audit table before attaching behavior

```
CREATE TABLE app.product_audit (  
  audit_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  product_id bigint NOT NULL,  
  action text NOT NULL,  
  old_stock integer,  
  new_stock integer,  
  changed_at timestamptz NOT NULL DEFAULT clock_timestamp(),  
  changed_by text NOT NULL DEFAULT current_user  
);
```

IMMUTABLE HISTORY

Store the before/after facts needed for investigation.

DESIGN CHOICE

Keep the actor, event time and affected key—not an unexplained text blob.

The trigger function records only meaningful stock changes

```
CREATE OR REPLACE FUNCTION app.audit_product_stock()
RETURNS trigger LANGUAGE plpgsql AS $function$
BEGIN
  IF OLD.stock_qty IS DISTINCT FROM NEW.stock_qty THEN
    INSERT INTO app.product_audit
      (product_id, action, old_stock, new_stock)
    VALUES
      (NEW.product_id, TG_OP, OLD.stock_qty, NEW.stock_qty);
  END IF;
  RETURN NEW;
END;
$function$;
```

PRECISE

No audit row when another product column changes.

TRANSACTIONAL

Audit insert commits or rolls back with the original update.

Attach, inspect and safely replace the trigger

```
DROP TRIGGER IF EXISTS trg_product_stock_audit  
ON app.products;
```

```
CREATE TRIGGER trg_product_stock_audit  
AFTER UPDATE OF stock_qty ON app.products  
FOR EACH ROW  
EXECUTE FUNCTION app.audit_product_stock();
```

```
\dS+ app.products
```

UPDATE OF

Limits firing to statements that target stock_qty.

WHEN CLAUSE

Can further prevent calls when OLD and NEW values are unchanged.

Keep trigger names descriptive; hidden behavior is already hard to discover.

Test trigger behavior inside a rollback-safe transaction

```
BEGIN;  
UPDATE app.products  
SET stock_qty = stock_qty - 1  
WHERE sku = 'KB-100';  
  
SELECT product_id, action, old_stock, new_stock, changed_by  
FROM app.product_audit  
ORDER BY audit_id DESC  
LIMIT 1;  
ROLLBACK;
```

EXPECTED

One UPDATE produces one matching audit row.

ROLLBACK PROOF

After rollback, neither stock nor audit history remains changed.

Test both the intended side effect and the no-change case.

Partitioning splits one logical table into physical tables

ONE LOGICAL INTERFACE

Queries target the partitioned parent. PostgreSQL routes rows and combines relevant partitions.

MANY PHYSICAL PARTITIONS

Each child owns a non-overlapping key range and can be indexed, detached or maintained separately.

Partitioning is a lifecycle and pruning tool—not an automatic speed button.

Choose the partition key from access and retention patterns

RANGE

Time or ordered intervals
Monthly order events
Supports rolling retention

LIST

Small known categories
Region or tenant group
Explicit value ownership

HASH

Even distribution by key
No natural ranges
Fixed modulus/remainder

For this lab: RANGE by occurred_at because queries and retention are time-based.

Create a partitioned event table with a valid key

```
CREATE TABLE app.order_events (  
  event_id bigint GENERATED ALWAYS AS IDENTITY,  
  order_id bigint NOT NULL,  
  event_type text NOT NULL,  
  occurred_at timestamptz NOT NULL,  
  payload jsonb,  
  PRIMARY KEY (event_id, occurred_at)  
+) PARTITION BY RANGE (occurred_at);
```

KEY RULE

A parent unique/primary key must include every partition-key column.

BOUNDARIES

Use non-overlapping lower-inclusive, upper-exclusive ranges.

The partition key must be present and cannot be updated across an invalid range.

Create explicit ranges plus a default safety net

```
CREATE TABLE app.order_events_2026_q3  
PARTITION OF app.order_events  
FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');
```

```
CREATE TABLE app.order_events_2026_q4  
PARTITION OF app.order_events  
FOR VALUES FROM ('2026-10-01') TO ('2027-01-01');
```

```
CREATE TABLE app.order_events_default  
PARTITION OF app.order_events DEFAULT;
```

DEFAULT

Prevents failed inserts when a new time range was not prepared.

OPERATIONS

Monitor default rows; move them before attaching a new overlapping partition.

Partition pruning works when the predicate exposes the key

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT event_id, order_id, event_type
FROM app.order_events
WHERE occurred_at >= TIMESTAMPTZ '2026-07-01'
      AND occurred_at < TIMESTAMPTZ '2026-10-01';
```

LOOK FOR

Only `order_events_2026_q3` appears in the plan.

AVOID HIDING THE KEY

Expressions and mismatched predicates can make pruning harder.

Verify pruning with EXPLAIN; do not assume the partition design works.

Full-text search compares lexemes—not raw substrings

1

PARSE

Tokenize text, normalize words and remove configuration-specific stop words.

2

MATCH

Compare a tsvector document with a tsquery using the @@ operator.

3

RANK

Order matches by relevance and optionally highlight the supporting text.

Choose the text-search configuration that matches the content language.

See how PostgreSQL normalizes documents and queries

```
SELECT to_tsvector('english',  
  'Wireless keyboards and keyboard cases');  
  
SELECT plainto_tsquery('english', 'wireless keyboard');  
SELECT phraseto_tsquery('english', 'wireless keyboard');  
SELECT websearch_to_tsquery('english',  
  '"wireless keyboard" -case');
```

TSVECTOR

Normalized lexemes plus positions and optional weights.

TSQUERY

Search terms connected by AND, OR, NOT, phrase or prefix operators.

websearch_to_tsquery accepts familiar user-facing search syntax.

Store the search vector and index it with GIN

```
ALTER TABLE app.products
ADD COLUMN IF NOT EXISTS description text;

ALTER TABLE app.products
ADD COLUMN search_vec tsvector
GENERATED ALWAYS AS (
  to_tsvector('english',
    coalesce(name, '') || ' ' || coalesce(description, ''))
) STORED;

CREATE INDEX products_search_gin
ON app.products USING gin (search_vec);
```

STORED VECTOR

Recomputed automatically when indexed source columns change.

GIN INDEX

Designed for membership queries such as lexeme containment.

Demo: search, rank and highlight catalogue results

```
WITH q AS (
  SELECT websearch_to_tsquery('english', 'wireless keyboard') AS query
)
SELECT p.sku, p.name,
       ts_rank(p.search_vec, q.query) AS rank,
       ts_headline('english', coalesce(p.description, ''), q.query) AS match
FROM app.products AS p
CROSS JOIN q
WHERE p.search_vec @@ q.query
ORDER BY rank DESC, p.sku;
```

FILTER

@@ returns only matching documents and can use the GIN index.

PRESENT

Rank orders relevance; headline shows why each row matched.

Hands-on: build all three capabilities

- 01 Create product_audit and its AFTER UPDATE trigger
- 02 Prove audit success and rollback behavior
- 03 Create order_events with Q3, Q4 and default partitions
- 04 Insert three dated events and identify their partitions
- 05 Add search_vec and the products_search_gin index
- 06 **Return ranked product matches with a highlighted excerpt**

PAIR CHECK AFTER TASK 3

Lab challenge: prove routing and pruning

```
INSERT INTO app.order_events(order_id,event_type,occurred_at) VALUES
(1,'PAID','2026-07-23 10:00+08'),
(1,'SHIPPED','2026-10-05 09:00+08'),
(2,'CREATED','2027-02-01 12:00+08');
```

```
SELECT tableoid::regclass AS physical_table, count(*)
FROM app.order_events
GROUP BY 1 ORDER BY 1;
```

ROUTING EVIDENCE

Rows land in Q3, Q4 and default partitions.

PRUNING EVIDENCE

EXPLAIN a Q3 date range and list scanned partitions.

Do not leave unexpected rows in the default partition unnoticed.

Validate the lab with six checks

```
\dS+ app.products  
\d+ app.order_events  
\di+ app.products_search_gin
```

```
SELECT * FROM app.product_audit ORDER BY audit_id DESC LIMIT 3;  
SELECT tableoid::regclass, count(*) FROM app.order_events GROUP BY 1;  
SELECT sku, name FROM app.products  
WHERE search_vec @@ websearch_to_tsquery('english','keyboard');
```

EXPECTED EVIDENCE

Audit trigger is listed
Audit values match the update
Three event partitions exist
Rows route to expected children
GIN index exists
Search returns relevant products

Troubleshoot by matching symptom to cause

SYMPTOM	LIKELY CAUSE	ACTION
record NEW has no field	Column name differs from trigger code	Inspect \d app.products
no partition found	No matching range or default	Create bounds or a DEFAULT partition
overlapping partition	New bounds conflict with a child/default row	Correct bounds and relocate rows
search returns no rows	Vector/config/query mismatch	Inspect vector and tsquery separately

Inspect the object definition and the exact failing row before rewriting code.

Knowledge check: choose the right database feature

SCENARIO A

Every stock change needs actor and before/after values.

Which trigger timing fits?

SCENARIO B

Five years of events are queried and retired by month.

Which partition strategy fits?

SCENARIO C

Users type phrases and expect relevant catalogue results.

Which index and operators fit?

Suggested: AFTER row trigger • RANGE by time • stored tsvector + GIN + @@.

Module 7 takeaway: make advanced features observable

1

BEHAVIOR

Keep trigger effects narrow, transactional and easy to inspect.

2

LIFECYCLE

Design partition keys and bounds around queries and retention.

3

RELEVANCE

Normalize, index, rank and explain what matched the user query.

Next: Module 8 — optimization, application connection and the final capstone.